IN403 · Advanced Network Protocols, Architectures & Security

LECTURE 03 · 2 HOURS

TCP Congestion Control II: CUBIC, BBR & Performance Optimization

High-speed congestion control, model-based BBR, AQM and ECN, and practical TCP tuning for real networks



Course topic: TCP/IP Deep Dive

Learning objectives

By the end of this session you will be able to:

1. Explain why Reno underperforms on long fat networks
2. Describe the CUBIC window function and its RTT-independence
3. Explain BBR’s model of bottleneck bandwidth and min-RTT and its state machine
4. Describe bufferbloat, AQM (CoDel, FQ-CoDel) and ECN/L4S
5. Apply TCP tuning techniques: buffers, pacing, offloads and algorithm selection

Session plan (120 minutes)



0:00 – 0:10	Recap: Reno limits on high-BDP paths
0:10 – 0:35	CUBIC: design and window function
0:35 – 1:00	BBR: model-based congestion control
1:00 – 1:10	Break
1:10 – 1:30	Bufferbloat, AQM and ECN
1:30 – 1:50	Performance optimization toolbox
1:50 – 2:00	Quiz and wrap-up

01

The high-BDP problem

Linear growth is too slow when windows are huge

Reno on a 10 Gb/s, 100 ms path

83 333

Segments needed in flight (BDP / 1500 B)

41 667

RTTs to grow back after halving (+1 per RTT)

≈ 70 min

Time to recover from a single loss

A single random loss every hour would keep Reno far below link capacity. High-speed variants (HSTCP, H-TCP, BIC, CUBIC) increase faster when far from the last loss point.

Reno on a 10 Gb/s, 100 ms path

EXPLAINED

On fast, long-distance links the window needed to fill the pipe is enormous. At 10 Gb/s and 100 ms RTT, the bandwidth-delay product is 125 MB, or about 83 000 full-size segments.

After a single loss Reno halves the window to about 41 700 segments and then adds only one segment per RTT. Climbing back takes about 41 700 RTTs — roughly 70 minutes. During that time the link is far from full. If losses occur more often than once an hour (random bit errors alone can do this), Reno can never reach full speed.

This is not a bug in Reno: its linear growth was designed for the small windows of the 1980s. The fix is to grow faster when the window is far below the point where loss last occurred, and to grow slowly near that point.

EXAMPLE

Compare: on a 10 Mb/s, 20 ms path the BDP is only 17 segments, so recovering from a loss takes about 8 RTTs = 160 ms. Same algorithm, completely different behaviour.

REMEMBER

Additive increase of +1 MSS per RTT does not scale with bandwidth × delay.

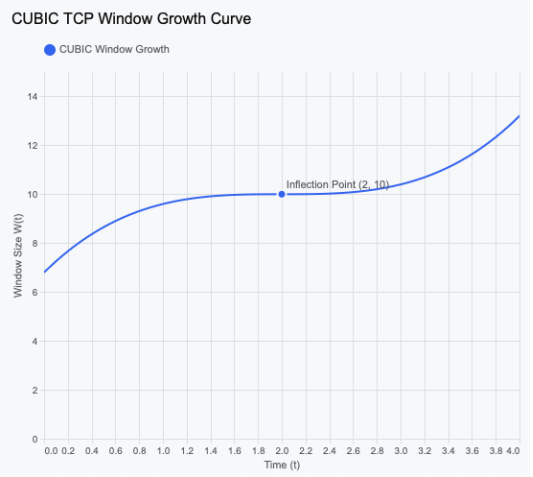
02

CUBIC

Default in Linux (since 2.6.19), Windows and macOS — RFC 9438

CUBIC window growth function

- **$W(t) = C \cdot (t - K)^3 + W_{max}$**
- **Parameters**
  - W_{max} — window just before the last reduction
  - t — time since the last reduction (not RTT count)
  - $C = 0.4$ scaling constant
  - $K = \sqrt[3]{W_{max} \cdot (1 - \beta)/C}$ — time to return to W_{max}
  - $\beta = 0.7$ multiplicative decrease (cwnd × 0.7 on loss)
- **Concave region:** grows fast after loss, slows as it approaches W_{max} (stable plateau)
- **Convex region:** beyond W_{max}, accelerates to probe for new capacity



3

RTT-independent growth

Because growth depends on wall-clock time since the last loss rather than on ACK arrivals, flows with different RTTs grow at the same rate — improving RTT fairness compared with Reno.

CUBIC window growth function

EXPLAINED

CUBIC replaces “+1 segment per RTT” with a **cubic function of the time since the last loss**. W_max is the window size just before the last loss; after a loss the window is reduced by 30% (β = 0.7) and then follows the curve $W(t) = C(t - K)^3 + W_{max}$.

The shape of a cubic is the key idea. Just after the loss the curve rises **steeply** (concave part), quickly recovering most of the lost window. As it approaches W_max it **flattens**, staying close to the level that previously caused loss — a stable plateau. If no loss occurs, the curve **accelerates again** above W_max (convex part) to probe for newly available bandwidth.

K is the time at which the curve returns to W_max. It depends only on W_max, β and C — not on the RTT. That is why CUBIC flows with different RTTs grow at similar speeds, making it fairer than Reno across long and short paths.

EXAMPLE

W_max = 1000 segments ⇒ after loss W = 700. $K = \sqrt[3]{(1000 \times 0.3 / 0.4)} \approx 9.1$ s. About 9 seconds later the window is back near 1000, regardless of whether the RTT is 10 ms or 200 ms.

REMEMBER

CUBIC: fast recovery far from W_max, caution near W_max, probing beyond it — all driven by time, not RTT.

CUBIC in practice

1 TCP-friendly region

CUBIC estimates what Reno would achieve and never goes below it — on short-RTT, low-BDP paths it behaves like Reno.

2 Fast convergence

If a loss occurs below the previous W_max, CUBIC lowers W_max further to release bandwidth for new flows.

3 HyStart++

Exits slow start early when RTT starts to rise (RFC 9406), avoiding massive overshoot and burst losses.

4 Still loss-based

CUBIC fills buffers until they overflow. In deep-buffered networks this creates high queueing delay.

CUBIC in practice

EXPLAINED

On short, low-latency paths a cubic curve could actually grow more slowly than Reno. CUBIC therefore computes the window that Reno would have and uses whichever is larger — the **TCP-friendly region** — so it never performs worse than standard TCP.

Fast convergence helps new flows: if a loss occurs before the window reaches the previous W_max, other flows have probably joined, so CUBIC lowers its W_max a little further and releases bandwidth sooner.

HyStart++ improves slow start. Instead of doubling until packets are lost (which can drop hundreds of packets at once on fast links), it watches the RTT; when RTT starts rising, queues are forming, so it leaves slow start early. However, CUBIC is still **loss-based**: in steady state it keeps filling buffers until they overflow, which causes queueing delay.

EXAMPLE

Linux, Windows and macOS all use CUBIC by default, so most TCP traffic you capture in the labs will show cubic-shaped window growth.

REMEMBER

CUBIC is the default almost everywhere, but it still needs loss to find the limit — and that fills queues.

03

BBR

Bottleneck Bandwidth and Round-trip propagation time (Google, 2016)

Loss-based vs model-based

Loss-based (Reno, CUBIC)

- Signal: packet loss
- Operates at the point where buffers overflow
- Throughput collapses with random (non-congestive) loss
- Causes bufferbloat in deep buffers

Model-based (BBR)

- Signal: measured delivery rate and RTT
- Targets Kleinrock's optimal point: rate = BtlBw, inflight = BDP
- Tolerant to random loss
- Paces packets rather than sending bursts

$$\text{Delivery Rate} = \frac{\text{Data delivered}}{\text{Time taken}}$$

$$\text{BDP} = \text{BtlBw} \times \text{RT}_{\text{min}}$$

Loss-based vs model-based

EXPLAINED

Loss-based algorithms use packet loss as the signal that the network is full. But a loss happens only after the bottleneck queue has overflowed — so these algorithms deliberately operate with **full buffers**. On networks with large buffers this adds hundreds of milliseconds of delay; on lossy wireless links, random losses are misread as congestion and throughput collapses.

BBR takes a different view: the network has a **bottleneck bandwidth** and a **minimum round-trip time**. The ideal operating point (Kleinrock's point) is to send exactly at the bottleneck rate with exactly one BDP in flight: the link is full but no queue builds up.

BBR measures these two quantities continuously from ACKs and sets its sending rate from its model, rather than reacting to every loss. It also **paces** packets evenly instead of sending bursts.

EXAMPLE

A Wi-Fi link loses 1% of packets due to interference, not congestion. CUBIC repeatedly cuts its window and gets a fraction of the capacity; BBR keeps sending at the measured delivery rate.

REMEMBER

Loss-based: "I am too fast when packets are dropped." Model-based: "I am too fast when I exceed the measured bottleneck rate."

BBR’s model and control parameters

BtlBw estimate	Windowed max of delivery rate over ~10 RTTs
RTprop estimate	Windowed min of RTT over ~10 s
Pacing rate	pacing_gain × BtlBw
cwnd cap	cwnd_gain × BtlBw × RTprop (≈ 2 × BDP)

- BBR sets **pacing rate** as the primary control; cwnd is only a safety cap
- Requires accurate delivery-rate sampling on every ACK
- Linux: **sysctl net.ipv4.tcp_congestion_control=bbr** with the fq qdisc for pacing

BBR’s model and control parameters

EXPLAINED

BBR estimates **BtlBw**, the bottleneck bandwidth, as the maximum delivery rate observed over the last few round trips. The delivery rate is measured per ACK: how many bytes were delivered divided by how long it took.

It estimates **RTprop**, the round-trip propagation time, as the minimum RTT seen over the last 10 seconds. The minimum is used because any higher value includes queueing delay.

From these, BBR computes its main control: the **pacing rate** (gain × BtlBw), which spaces packets evenly in time. The congestion window is only a safety cap of about two BDPs (2 × BtlBw × RTprop) to handle delayed or stretched ACKs. On Linux, BBR relies on the fq queueing discipline to perform the pacing.

EXAMPLE

Measured BtlBw = 100 Mb/s, RTprop = 40 ms ⇒ BDP = 500 KB. BBR paces at about 100 Mb/s and caps inflight at about 1 MB.

REMEMBER

BBR: rate = max recent delivery rate; delay floor = min recent RTT; send at that rate with about one BDP in flight.

BBR state machine



STARTUP

Pacing gain ≈ 2.89; doubles rate each RTT until bandwidth stops growing by 25% for 3 rounds.

DRAIN

Inverse gain (≈0.35) to empty the queue built during STARTUP until inflight ≈ BDP.

PROBE_BW

Steady state; cycles gains 1.25, 0.75, then 1 × 6 to probe for more bandwidth and drain.

PROBE_RTT

Every ~10 s, cut inflight to 4 packets for ~200 ms to re-measure RTprop.

BBR state machine

EXPLAINED

STARTUP behaves like slow start: it increases the pacing rate by a gain of about 2.89 each round to discover bandwidth quickly. When three rounds pass without a 25% increase in measured bandwidth, the pipe is full.

STARTUP overshoots and creates a queue, so **DRAIN** temporarily sends slower than BtlBw until inflight falls back to one BDP. Then BBR enters **PROBE_BW**, where it spends most of its life: it cycles through gains of 1.25 (probe for more bandwidth), 0.75 (drain any queue created) and 1.0 for six rounds (cruise).

Because the minimum RTT can only be measured when queues are empty, **PROBE_RTT** periodically (about every 10 s) reduces inflight to just 4 packets for around 200 ms, then returns to PROBE_BW.

EXAMPLE

In Lab 2 you can see PROBE_RTT as a short dip in throughput roughly every 10 seconds in the iPerf3 interval output.

REMEMBER

STARTUP → DRAIN → PROBE_BW (steady cycling) with periodic PROBE_RTT to refresh the minimum RTT.

CUBIC vs BBR at a glance

Property	CUBIC	BBR v1	BBR v2/v3
Primary signal	Loss	Rate + RTT	Rate + RTT + loss/ECN
Queue occupancy	High (fills buffer)	Low	Low
Random-loss tolerance	Poor	Very high	High (bounded)
Fairness vs CUBIC	Baseline	Can starve CUBIC in shallow buffers	Improved
Deployment	Default in most OSes	Google, YouTube, CDNs	Google internal, Linux patches

CUBIC vs BBR at a glance

EXPLAINED

The two algorithms answer different questions. CUBIC uses loss and therefore keeps queues full; BBR uses rate and RTT and keeps queues short. BBR tolerates random loss much better, which is why it is popular for video streaming and long-distance transfers.

BBR version 1 had a fairness problem: in networks with shallow buffers it could take far more than its share from CUBIC flows, and it ignored loss, causing high retransmission rates. **BBRv2 and v3** add loss and ECN as secondary signals and limit how much data they keep in flight when losses rise.

In practice, the choice depends on the environment: CUBIC is a safe default, BBR suits lossy or long-haul paths where you control the sender (for example a content provider), and DCTCP suits data centres with ECN-capable switches.

EXAMPLE

Google reported significant throughput and latency improvements on YouTube and Google Cloud after switching to BBR on their servers.

REMEMBER

No single algorithm is best everywhere. Match the algorithm to the network and measure.

04

Bufferbloat, AQM and ECN

The network side of congestion control

Bufferbloat and Active Queue Management

- **Bufferbloat:** oversized FIFO buffers + loss-based TCP ⇒ seconds of queueing delay under load
- **Drop-tail** signals congestion too late and synchronises flows
- **RED:** drops early with probability rising with average queue length — hard to tune
- **CoDel:** drops when packet sojourn time exceeds 5 ms for 100 ms — parameterless in practice
- **FQ-CoDel / CAKE:** per-flow queues + CoDel; default qdisc in many Linux distributions and routers

Q

Measure it

Run a latency test (ping) while saturating the uplink with iPerf3. A jump from 20 ms to 500+ ms indicates bufferbloat. Switching to fq_codel usually restores low latency.

Bufferbloat and Active Queue Management

EXPLAINED

Routers and home gateways often have very large buffers, because memory is cheap and vendors want to avoid drops. Combined with loss-based TCP, which fills buffers until they overflow, this produces **bufferbloat**: under load, every packet waits behind hundreds of milliseconds or even seconds of queued data.

Plain FIFO with tail drop signals congestion only when the buffer is full. **RED** drops packets randomly before the queue fills, but its thresholds are hard to tune. **CoDel** measures how long each packet waited (sojourn time); if the minimum wait stays above 5 ms for 100 ms, a standing queue exists and CoDel starts dropping.

FQ-CoDel and **CAKE** also separate flows into their own queues and serve them in turn, so a bulk download cannot delay a small interactive flow. These are default in many Linux distributions and in modern home routers.

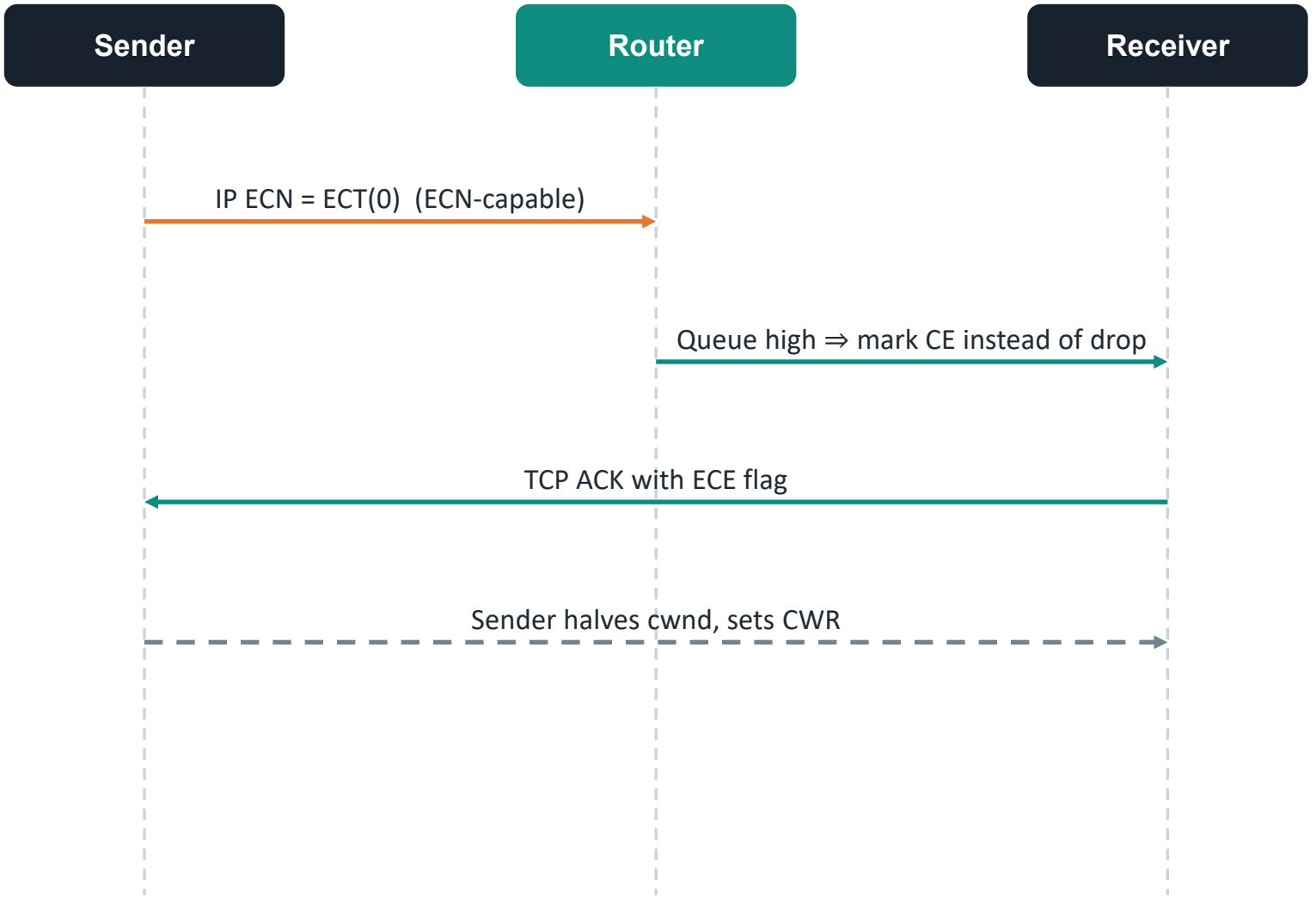
EXAMPLE

Ping your router while uploading a large file: 5 ms idle, 800 ms during upload = bufferbloat. Enabling fq_codel (or SQM on OpenWrt) typically brings it back to 10–20 ms.

REMEMBER

Large dumb buffers + loss-based TCP = high latency. AQM keeps queues short; fair queueing isolates flows.

Explicit Congestion Notification (RFC 3168)



- Congestion signalled **without losing packets**
- Negotiated in the handshake (SYN with ECE+CWR)
- **DCTCP** reacts in proportion to the fraction of marked packets
- **L4S** (RFC 9330–9332): scalable CC + shallow ECN marking for sub-ms queueing
  - Reacts to each CE mark with a **small, proportional** window/rate trim (α/2 style, like DCTCP) — *not a halving*
  - Routers mark CE at **~1ms queueing delay or less**

Explicit Congestion Notification (RFC 3168)

EXPLAINED

Dropping packets to signal congestion wastes the work already done to carry them and forces retransmission. **ECN** lets a router signal congestion by **marking** packets instead of dropping them.

Senders that support ECN set the IP header's ECN field to "ECN-capable" (ECT). When a router's AQM decides a packet should signal congestion, it changes the field to **CE** (Congestion Experienced) and forwards the packet. The receiver echoes this back to the sender with the **ECE** flag in TCP ACKs; the sender reduces cwnd as if a loss had occurred and confirms with the **CWR** flag.

In data centres, **DCTCP** reduces cwnd in proportion to the fraction of marked packets, keeping queues tiny. The newer **L4S** architecture brings the same idea to the Internet, aiming for sub-millisecond queueing delay.

EXAMPLE

Wireshark: an IP packet with "ECN: CE (3)" followed by a TCP ACK with the ECE flag set, then a data segment with CWR set.

REMEMBER

ECN = congestion signal without loss. Both endpoints and the router's AQM must support it.

05

Performance optimization

Getting the most out of TCP on real hosts and networks

TCP performance toolbox

1 Size buffers to BDP

Raise rmem/wmem maxima so autotuning can reach the BDP; check window scaling is enabled end-to-end.

2 Choose the algorithm

CUBIC for general use; BBR for lossy or long-haul paths; DCTCP inside data centres with ECN.

3 Pace traffic

fq qdisc with pacing smooths bursts and reduces loss at shallow-buffered switches.

4 Offloads

TSO/GSO and GRO/LRO cut per-packet CPU cost; checksum offload; multi-queue NICs with RSS (**Receive Side Scaling)**

5 Reduce handshakes

Connection reuse, TCP Fast Open, TLS 1.3 / 0-RTT, larger initial window.

6 Avoid PMTU black holes

Enable PLPMTUD (tcp_mtu_probing) and MSS clamping on tunnels.

TCP performance toolbox

EXPLAINED

Most real-world TCP performance problems fall into a few categories. If buffers are smaller than the BDP, a single flow can never fill the path — raise the maximum socket buffers. If the path is lossy or very long, a different congestion control algorithm may help.

Bursty traffic overflows shallow switch buffers; **pacing** (the fq qdisc) spreads packets evenly. At high speeds the CPU becomes the limit, so **offloads** (TSO, GRO, checksum offload) and multi-queue NICs let the hardware do per-packet work.

For short transfers, the number of round trips dominates: reuse connections, use TCP Fast Open, TLS 1.3 and a larger initial window. Finally, tunnels (VPN, GRE) reduce the MTU; if ICMP “packet too big” messages are blocked, connections hang — **MTU probing** and **MSS clamping** fix these black holes.

EXAMPLE

A VPN user can load small pages but large downloads stall: classic PMTU black hole. Clamping the MSS on the VPN gateway to 1360 bytes solves it.

REMEMBER

Identify the limit first (window, loss, CPU, RTTs, MTU), then apply the matching fix.

Linux tuning cheat-sheet

```
# Available / current congestion control
sysctl net.ipv4.tcp_available_congestion_control
sysctl -w net.ipv4.tcp_congestion_control=bbr

# Fair-queue qdisc with pacing
tc qdisc replace dev eth0 root fq

# Buffers for a 10 Gb/s x 100 ms path (~125 MB)
sysctl -w net.core.rmem_max=134217728
sysctl -w net.core.wmem_max=134217728
sysctl -w net.ipv4.tcp_rmem="4096 87380 134217728"
sysctl -w net.ipv4.tcp_wmem="4096 65536 134217728"

# Path MTU probing and ECN
sysctl -w net.ipv4.tcp_mtu_probing=1
sysctl -w net.ipv4.tcp_ecn=1

# Offload status
ethtool -k eth0 | grep -E "tcp-seg|generic-rec"
```

- Change one thing at a time and measure
- Per-socket override: setsockopt TCP_CONGESTION
- Changes via sysctl -w are not persistent — use /etc/sysctl.d/

Linux tuning cheat-sheet

EXPLAINED

Linux lets you change congestion control system-wide with a sysctl or per socket with the TCP_CONGESTION option; iperf3 -C uses the latter. Before switching to BBR, load the module and install the **fq** qdisc so packets can be paced.

Buffer settings come in pairs: **net.core.rmem_max / wmem_max** are hard ceilings for all sockets, while the third value of **tcp_rmem / tcp_wmem** is the maximum that auto-tuning may reach. Both must exceed the BDP of your longest high-speed path.

tcp_mtu_probing helps when PMTU discovery is broken, and tcp_ecn=1 requests ECN on outgoing connections. Changes made with sysctl -w are lost at reboot; persistent settings go in /etc/sysctl.d/. Always change one parameter at a time and measure before and after.

EXAMPLE

10 Gb/s × 100 ms = 125 MB BDP ⇒ set rmem_max and wmem_max to at least 134 217 728 (128 MB).

REMEMBER

Tune with evidence: measure, change one setting, measure again, and document.

Choose the right congestion control

IN-CLASS ACTIVITY · 12 MIN

For each scenario, pick an algorithm and justify it in one sentence.

- A CDN serving video over intercontinental links with 1% random wireless loss at the edge
- A data-centre storage cluster with ECN-capable switches and 10 µs RTTs
- A university campus with mixed traffic and a deep-buffered internet uplink
- A satellite link with 600 ms RTT and 50 Mb/s capacity

Choose the right congestion control

IN-CLASS ACTIVITY · 12 MIN

- A CDN serving video over intercontinental links with 1% random wireless loss at the edge
  - **BBR (Bottleneck Bandwidth and Round-trip propagation time)**.
  - **Reasoning:** BBR is model-based and measures bandwidth and RTT directly rather than reacting to packet loss as a sign of congestion. In wireless environments with random loss, loss-based algorithms (like CUBIC) would unnecessarily throttle their windows, whereas BBR maintains high throughput.
- A data-centre storage cluster with ECN-capable switches and 10 µs RTTs
  - **DCTCP (Data Center TCP) with Explicit Congestion Notification (ECN)**.
  - **Reasoning:** DCTCP leverages ECN to mark packets early when queues start building up slightly, reacting proportionally to the extent of congestion rather than waiting for dropped packets. This keeps latencies extremely low in ultra-low RTT data center environments.

Choose the right congestion control

IN-CLASS ACTIVITY · 12 MIN

- A university campus with mixed traffic and a deep-buffered internet uplink
  - **CUBIC or BBR with FQ-CoDel (Fair Queuing Controlled Delay)**.
  - **Reasoning:** Deep buffers cause "bufferbloat," leading to high latency spikes. Using an active queue management algorithm like CoDel or FQ-CoDel helps manage queue lengths and drop/mark packets before latency gets out of hand, while CUBIC handles the mix of standard long-lived and short-lived campus traffic.
- A satellite link with 600 ms RTT and 50 Mb/s capacity
  - **BBR**.
  - **Reasoning:** Satellite links have a very high Bandwidth-Delay Product (BDP) due to the large 600 ms RTT. Standard algorithms struggle to scale up their congestion windows quickly enough over high-delay paths (BBR handles high-BDP paths efficiently).

Check your understanding

Q1 Why is CUBIC’s window growth independent of RTT, and why does that help fairness?

Q2 What two quantities does BBR estimate, and how does it measure each?

Q3 Explain how bufferbloat arises and how FQ-CoDel mitigates it.

Q4 How does ECN let a router signal congestion without dropping packets?

Model answers

A1 Its window is a function of the time elapsed since the last loss, not of the number of ACKs received. Flows with long and short RTTs therefore grow at the same pace, instead of short-RTT flows winning as in Reno.

A2 BtlBw (bottleneck bandwidth) = maximum recent delivery rate, measured from ACKs (bytes delivered / time). RTprop = minimum RTT seen recently (≈10 s window), refreshed in PROBE_RTT.

A3 Large FIFO buffers + loss-based TCP keep queues full, adding long delay for everyone. FQ-CoDel drops (or marks) when packets wait too long and gives each flow its own queue, so delay stays low and flows are isolated.

A4 The router sets the CE codepoint in the IP header instead of dropping; the receiver echoes ECE in TCP ACKs; the sender reduces cwnd and replies with CWR.

Key takeaways

- Reno’s +1 MSS per RTT is far too slow for high-BDP paths
- CUBIC grows as a cubic function of time since the last loss, with $\beta = 0.7$
- BBR models bottleneck bandwidth and min-RTT and paces at the estimated rate
- AQM (CoDel, FQ-CoDel) and ECN reduce queueing delay and loss
- Tune buffers to the BDP, enable pacing and measure every change

Reading & next session: RFC 9438 (CUBIC) · Cardwell et al., “BBR”, ACM Queue 2016 · RFC 8289 (CoDel) · Next: Lecture 4 — QoS Fundamentals & Traffic Shaping